Yachay Tech University
School of Mathematical and Computational Sciences
Course: Distributed Systems | Workshop 6

Workshop 6

Distributed Middleware with RabbitMQ

Activity 1: C/Python Comparison
Activity 2: Federated Learning Architecture

Student: Dario Pomasqui

Course: Distributed Systems – Semester II 2026

Instructor: Prof. Francisco Hidrobo

October 22, 2026

Workshop 6 project repository on GitHub

Abstract

This report documents both Workshop 6 activities. Activity 1 runs the RabbitMQ point-to-point queue and fanout examples in Python, then installs `librabbitmq-dev` in a reproducible Ubuntu environment to build and run the C producer and consumer. Activity 2 designs and implements a RabbitMQ communication prototype for the federated learning project on leaf disease detection. Three farm processes exchange instructions and updates; the coordinator computes FedAvg with all clients and with a partial quorum. Tests validate message transport, acknowledgments, and aggregation using synthetic values. They do not train a CNN or measure classification accuracy.

1. Context and Objectives

The project proposal compares FedAvg and FedProx for leaf disease classification using non-IID data distributed across Farm A, Farm B, and Farm C. Each farm keeps its images locally and performs local training. A coordinator receives parameter updates, aggregates them according to sample counts, and manages training rounds.

The initial proposal described RPC with Flower without specifying queues, delivery confirmations, or failure recovery. For this submission, the architecture uses RabbitMQ over AMQP 0-9-1 and the Pika Python client. The project proposal source at `../../proporsal/proposal.tex` was updated with this design and distinguishes model synchronization from distributed consensus.

2. Activity 1: RabbitMQ and Client Comparison

2.1 What the Activity 1 Folder Contains

I organized the supplied examples by communication pattern and implementation language. The `python-basic/` folder contains `producer.py`, `consumer.py`, and their Pika dependency list; these files demonstrate point-to-point messaging through the `hello` queue. The `python-pubsub/` folder contains `pub.py`, `sub.py`, and its requirements file; these demonstrate fanout publication to multiple subscribers. The `c-basic/` folder contains the C producer, consumer, and Makefile used to build the AMQP clients.

I added `../solution1/resolucion-actividad1/` to document and reproduce my work. It contains this activity's Markdown report, the PowerShell test script, a Dockerfile for the C build environment, and `evidencias/` with broker screenshots, command output, and inspection notes. The original examples remain in their respective folders; the added folder records how I tested them and what I observed.

2.2 Environment and Procedure

- I started the `rabbitmq:3-management` Docker image as my broker (RabbitMQ 3.13.7). I used AMQP port 5672 for clients and port 15672 for the Management UI.
- I used Python 3.13.14 with Pika 1.4.4 and ran the four supplied Python programs: the queue producer and consumer, and the fanout publisher and subscriber.
- To compile the C examples, I built the Docker image `workshop6-c-dev` on Ubuntu 24.04 with `build-essential` and `librabbitmq-dev` 0.11.0-1ubuntu0.2. I then used the supplied Makefile to build both programs.
- I also updated the C examples to accept an optional `RABBITMQ_HOST` value and to check connection, channel, and publish errors.

I recorded the repeatable steps in `../solution1/resolucion-actividad1/ejecutar-pruebas.ps1`. I can run the script from that folder after starting the broker; it requires Docker Desktop and Python:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\ejecutar-pruebas
    .ps1
```

2.3 Tests and Results

Test	Result	Observation
Pika AMQP connection	PASS	Connected to `localhost:5672`.
Python queue	PASS	`Hello Middleware!` published, received, and acknowledged; durable queue `hello`, QoS 1.
Python fanout	PASS	Both exclusive queues received `Workshop6 fanout test`.
Python syntax	PASS	All four scripts passed `py_compile`.
C compilation	PASS	`make` built producer and consumer, linking `-lrabbitmq`.
C queue	PASS	Consumer received `Hello from C!` and sent an ack.

Table 1: Activity 1 functional tests.

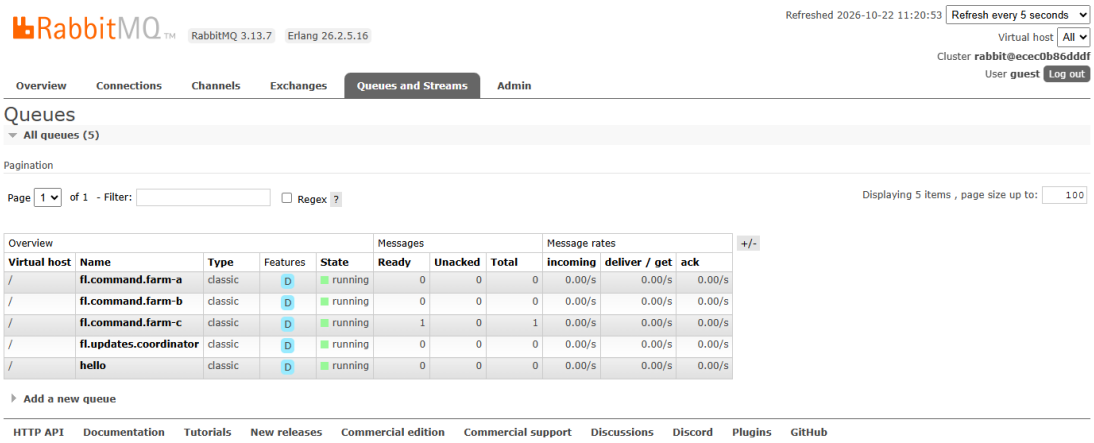


Figure 1: RabbitMQ Management: durable `hello` queue with no pending messages after the test.

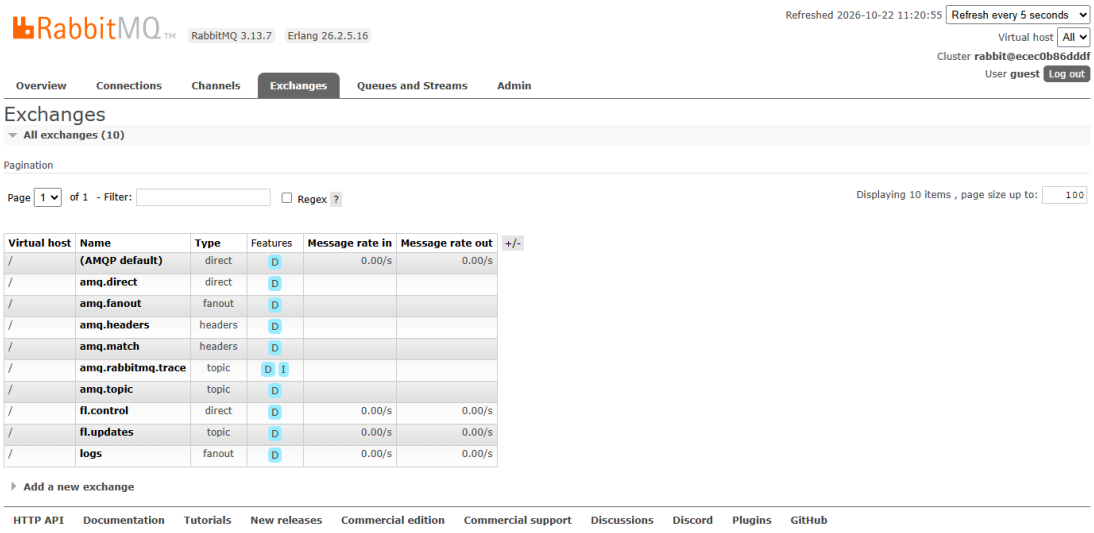


Figure 2: Observed exchanges: example `logs` fanout exchange and Activity 2 exchanges.

2.4 C and Python Comparison

Aspect	Python (Pika)	C (rabbitmq-c)
Interface	High-level API and callbacks; few lines are needed to publish and consume.	Explicit control of sockets, connections, channels, bytes, and message envelopes.
Dependencies	Python interpreter and Pika package.	Compiler, Make, headers, and library; installed here in Ubuntu Docker.
Delivery	Durable queue, persistent message, QoS 1, and manual acknowledgment.	Durable queue, persistent message, and manual acknowledgment.
Suitability	Straightforward integration with PyTorch and Python code.	Suitable for native components or existing C systems.

Table 2: Implementation comparison.

By running both clients against the same AMQP broker, I confirmed that the language does not change the queue behavior. For this machine learning project, I find Python easier to integrate with the model code. C gives me more low-level control, but I need to manage its compiler and native library dependencies.

3. Activity 2: Middleware for the Project

3.1 Architecture

The topology is hub-and-spoke with a **single logical writer** for the global model. RabbitMQ routes and stores messages; the coordinator keeps round state and computes aggregations. Each farm is an independent process with private images and a local model copy.

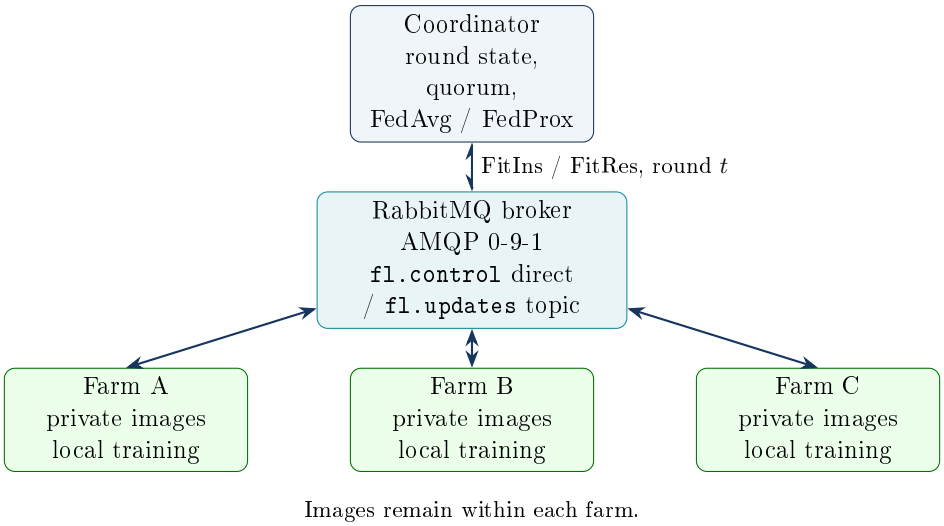


Figure 3: Architecture tested with local processes. Instructions and results flow through the broker; deployment will connect remote nodes using TLS.

3.2 Exchanges, Queues, and Messages

Flow	RabbitMQ	Purpose
Coordinator → farm	Durable direct exchange `fl.control`; queue `fl.command.<farm-id>` per node	`FitIns`: round identifiers, global model, and configuration.
Farm → coordinator	Durable topic exchange `fl.updates`; queue `fl.updates.coordinator`, binding `fit.*.*`	`FitRes`: round, client, sample count, parameters, and metrics.
Future observability	Topic exchange `fl.events`	Completed rounds, timeouts, and failures.

Table 3: AMQP routing.

Messages include `message_id`, `correlation_id=round_id`, type, and schema version. Farms use manual acknowledgments and publisher confirms before acknowledging an instruction. The coordinator validates round and client identifiers, discards duplicates and stale results, and counts each client only once.

3.3 Failures, Consistency, and Rationale

AMQP delivery can be repeated, so the application performs deduplication and does not claim exactly-once delivery. The coordinator completes a round when the configured quorum is reached or the deadline expires. Only then does it publish the next global version. If quorum is not reached, it retains the previous version. Disconnected clients keep instructions in durable queues and must install the current version when reconnecting.

FedAvg is not a consensus protocol such as Raft or Paxos. The coordinator is a single logical writer; tolerating its failure requires durable checkpoints and leader election or fencing. Replicating only the broker does not prevent two active coordinators. During a partition, local inference may continue, but a global model is not committed without the required update threshold.

RabbitMQ decouples local training time from coordinator availability, routes results by key, and stores work while a client is disconnected. A direct exchange routes instructions to each farm, while a topic exchange collects results. Pika and rabbitmq-c both support the protocol, enabling interoperability.

The broker alone does not provide differential privacy, secure aggregation, or Byzantine fault tolerance. Deployment requires TLS, per-node credentials and virtual hosts, resource limits, payload validation, and protection for model updates. CNN tensors should use an efficient binary format; JSON is used only in this prototype.

3.4 Functional Tests

Processes exchanged synthetic scalar updates to validate transport and FedAvg averaging; the dataset was not loaded and PyTorch was not used for training.

Test	Inputs	Result
Three farms	Initial weight 0; A: $n = 10, w = 1$; B: $n = 20, w = 2$; C: $n = 30, w = 4$.	A/B/C published and received acknowledgments. $w = (10 + 40 + 120)/60 = 2.8333$; `quorum_complete`.
Farm C offline	A/B active; minimum quorum 2.	A/B contributed; C's instruction remained pending in its durable queue. $w = (10 + 40)/30 = 1.6667$; `partial_quorum`.
Unit tests	Averaging, incompatible dimensions, and non-positive sample counts.	3 of 3 passed.

Table 4: Activity 2 tests.

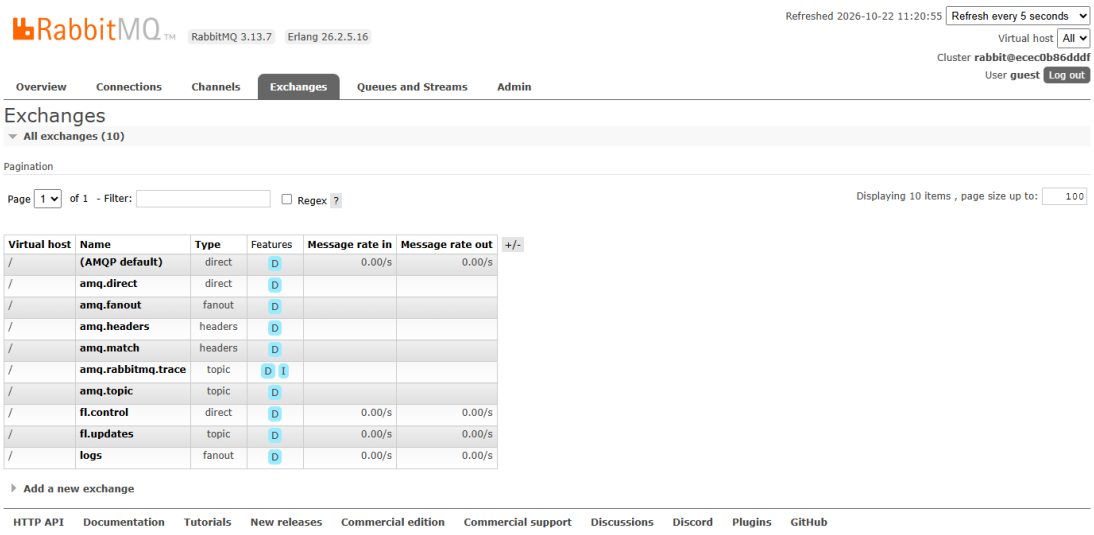


Figure 4: Live exchanges: `fl.control` direct and `fl.updates` topic.

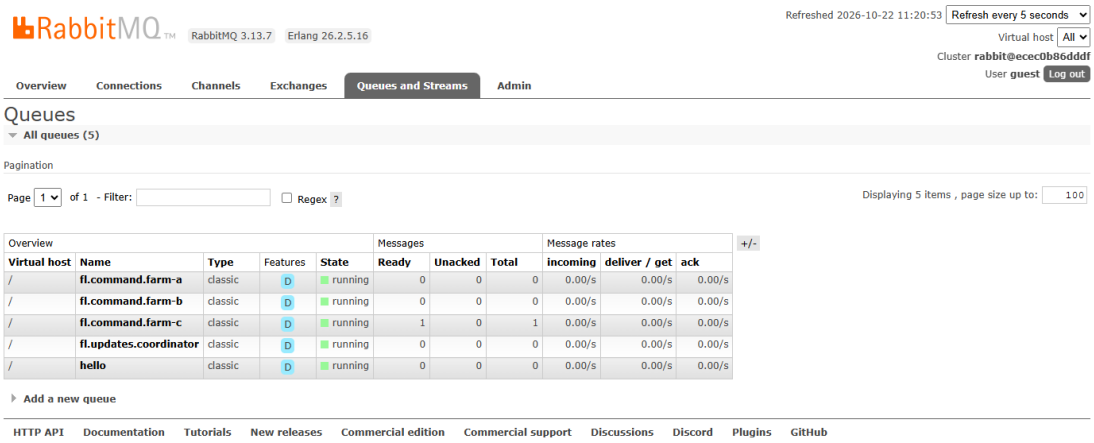


Figure 5: Farm C offline: its durable instruction remains pending (Ready=1).

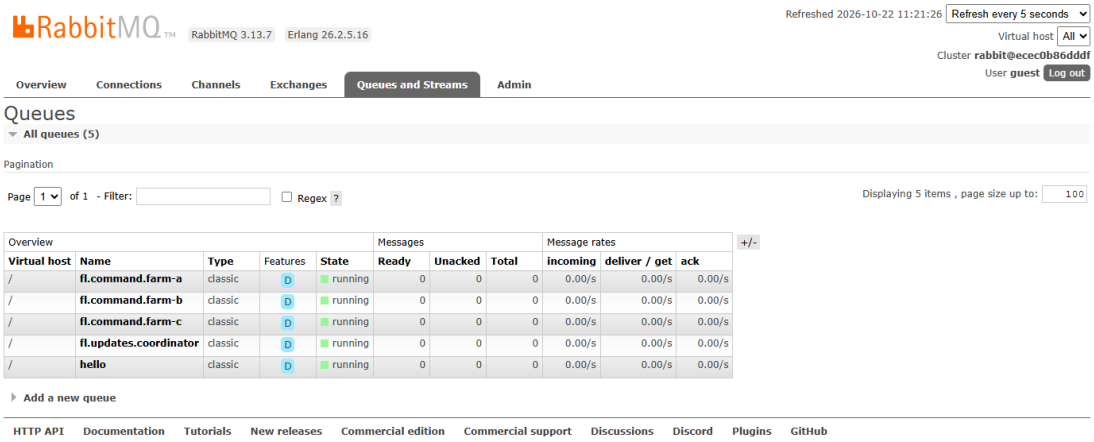


Figure 6: After the complete round, all five queues are empty with no unacknowledged messages.

4. Discussion and Limitations

From these tests, I confirmed that my C and Python clients interoperate with the same broker, that acknowledgments clear delivered messages, and that fanout reaches multiple subscribers. I also observed how a durable queue retains Farm C's instruction while that client is disconnected.

The prototype does not load PlantVillage, train with PyTorch, serialize CNN tensors, simulate a WAN, or implement coordinator failover or TLS. Demo deduplication is in memory. The values 2.8333 and 1.6667 verify a formula with synthetic inputs; they are not accuracy, F1, or FedAvg/FedProx convergence results.

5. Conclusions

Activity 1 was completed for Python and C after installing `librabbitmq-dev` in the Ubuntu image. For the project, a RabbitMQ path was implemented between a coordinator and three farm processes, with persistent instructions, correlated results, and FedAvg aggregation. Next steps are to replace scalar deltas with PyTorch tensors, persist checkpoints and transactional deduplication, measure latency and payload behavior under partitions, and integrate FedProx and classification metrics.

Submitted Work

For Activity 1, I documented the RabbitMQ examples, the steps I followed to run them, the test results, and my comparison of the C and Python clients. I included the test script and screenshots as supporting evidence. For Activity 2, I implemented a RabbitMQ prototype for communication between the coordinator and the farm clients. I included the source code, unit test results, sample aggregation outputs, and screenshots of the broker. I also updated the project proposal to describe the distributed communication design.